

This report aims to analyze and reflect upon the testing activities undertaken on the Online Bookstore project. The assignment asked for the design of a test suite to test selected features of the application, together with a written rationale to explain the design choices. To complete this task, I submitted Pytest scripts written in an automated way and a report that formalizes the testing plan. These two elements show mastery of technical skills and a systematic approach towards thinking through and reporting on the project's functional and non-functional requirements.

The automated tests were created to examine the core user interactions of an online bookstore, such as browsing, searching for books, managing books in a shopping cart, performing a checkout, and user account operations. I chose these tasks for testing because they are the most important part of the customers' experience. If they did not work right, the system would not be fit for a customer. The test verification went from broad tests to more narrow tests and from high-level checks to testing specific scenarios. At the outset, tests were projected to check that routes and pages would open reliably. Functional tests were designed to test functions like adding and removing books from the cart, as well as checking out. Here, I found opportunities for negative cases to be tested, like when a user searches for a book with no search terms or an invalid category search. I tested those cases to ensure the application could handle the error without making the application unusable for the user.

The automated tests were designed to test the fundamental interactions of an online bookstore: browsing and searching for books, managing items in the shopping cart,

checking out, and managing user accounts. I chose these functionalities since they are core to the customer experience. Any failure of these functionalities would affect the usability of the system. The design approach moved through relative stages from general checks to specific scenarios. The first tests guaranteed that routes and pages could be expected. function tests guaranteed the operations worked for examples, like adding or removing books to/from the cart and completing the checkout transaction. Negative cases were also included in the tests, like searches that had empty queries or selected invalid categories, to confirm the application failed gracefully.

The requirements for the Online Bookstore include reliable browsing, cart, and checkout operations, and the checkout process is secure. The automated tests have demonstrated that category and keyword searches produce relevant results, and that cart totals are updated as quantities change. This structured document builds upon the test results by detailing the discount code scenario, verifying payment failure scenarios, and sending order confirmation emails. It also covers account functionality such as logging in, logging out, and viewing order histories, so that both user-facing and administrative functionality have been tested.

Upon reflection, one significant strength was the interplay of practical and theoretical approaches. The Pytest suite provided fast feedback and ensured that the tests that were implemented passed, which gave confidence in the application's correctness. Simultaneously, the structured testing document provided linkage between objectives and test cases and specified expected outcomes as part of traceability. One of the

limitations of the project was that in some of the forms, there was little server-side validation, and placeholder data was utilized. However, the document recommends future changes for stricter input checks, heightened security scenarios, and using load tests that better mimic a real-life scenario. This demonstrated an awareness of future development needs and identified other areas where the testing plan could evolve.

To sum up, the project presents a complete and balanced testing strategy. The automated Pytest suite demonstrates functionality via the working scripts, and the structured testing document provides systematic coverage of functional and non-functional requirements. Collectively, they demonstrate alignment with the rubric evidenced through maintainable and modular coding, successful tests, transparent rationale, and reflection. The result is a submission that demonstrates not only the ability to design and execute useful tests, but also the ability to critically reflect on tests and their purpose in ensuring quality software.